Chapter 5
Wallets

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content



Wallet Technology Overview

Nondeterministic (Random) Wallets

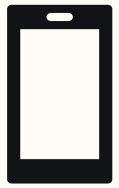
Deterministic (Seeded) Wallets

HD Wallets (BIP-32/BIP-44)

Seeds and Mnemonic Codes (BIP-39)

Wallet Best Practices

Using a Bitcoin Wallet



Wallet Technology Details

Mnemonic Code Words (BIP-39)

Creating HD Wallet from the Seed

Using an Extended Public Key on a Web Store

3 Wallets



Semantic

What is «Wallet» in Block Chain?



Items Controlled

What is in control of «Wallet»

4 Wallets



The word Wallet

is used to describe
a few different things
in bitcoin



At a high level

application
the primary user interface



programmer’s perspective

the data structure
to store and manage a user’s keys

5 Wallet Usecases

- access to a user's money
- managing keys and addresses
- tracking the balance
- Create and sign transactions



Nondeterministic (Random) Wallets



Deterministic (Seeded) Wallets



HD Wallets (BIP-32/BIP-44)



Seeds and Mnemonic Codes (BIP-39)



Wallet Best Practices



Using a Bitcoin Wallet

6

Wallet Technology Overview

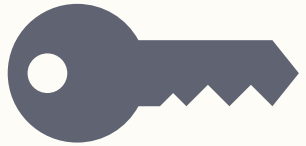
7 Wallet Technology Overview



Misconception

«bitcoin wallets

contain bitcoin»



In fact

the wallet

only contains keys

Wallet Technology Overview



Where are «coins»?

The “coins” are recorded
on the bitcoin network
in the blockchain



Users control the coins

by signing transactions
with the keys
in their wallets



Wallet = Keychain

bitcoin wallet is a keychain
Keeps chain of keys

Wallet Technology Overview



Keys not coins

Bitcoin wallets contain keys,

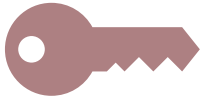
not coins



Wallet for User

Each user has a wallet

containing keys



Wallets are keychains

keeping pairs of

private/public keys

10 Wallet Technology Overview



Sign with Keys

Users sign transactions

with the keys



Prove of Ownership

the transaction outputs

can be spent



Transaction outputs

The coins are stored on the blockchain

in transaction outputs



TXOUT

Transaction Ouptput

often noted as vout

or txout

11

Types of Wallet vs Key Relations



2 Wallet Types

There are two primary types of wallets



Deterministic

keys are related to each other



Nondeterministic

keys are **not** related to each other

12 *Nondeterministic or* JBOK wallet



Nondeterministic wallet

Historically older



Features

each key is independently generated

from a random number.



JBOK wallet

Alias of nondeterministic wallet



Abbreviation

Just a Bunch Of Keys

13 Deterministic Wallet



Seed

master key

to derive other keys from



Regeneration

keys

can be generated again

from the original seed



Keys are related

All the keys in the wallet

are related to each other

Deterministic Wallet



HD Wallet

Hierarchical

Deterministic



The most common

derivation method



Tree structure

Consistute a tree-like structure



Seed

Deterministic wallets are initialized

from a seed.



Mnemonic

seeds are encoded as words

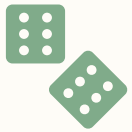
for easier use use

Nondeterministic (Random) Wallets



Historic

Bitcoin Core: the first bitcoin wallet



Random keys

wallets were collections of

randomly generated private keys



100 Keys

Bitcoin Core client pre-generates

100 private keys randomly at the start



New keys

generated as needed,

using each key only once

Nondeterministic (Random) Wallets



Replaced

Random wallets are replaced with deterministic wallets



Reasons

Hard to manage

back up

import

Nondeterministic (Random) Wallets: Disadvantages



Many keys to backup

keep copies of all keys

backed up wallet frequently



Risk

if wallet becomes inaccessible

the funds are irrevocably lost

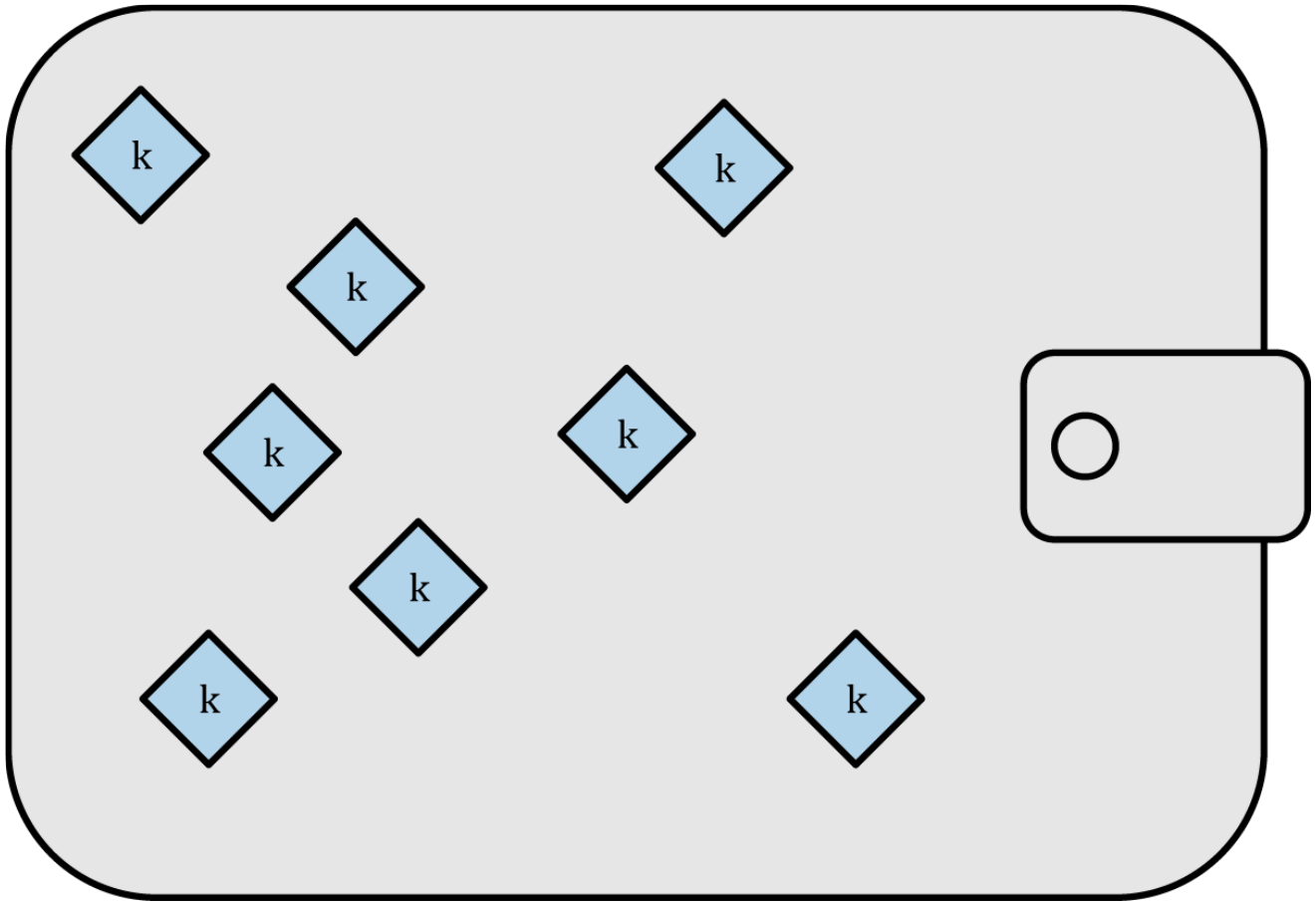


Figure 5-1. Type-0 nondeterministic (random) wallet: a collection of randomly generated keys

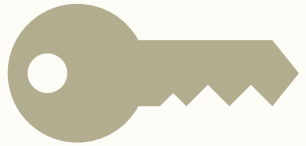
Deterministic (Seeded) Wallets



Wallets with Seed

contain private keys

derived from a common seed



one-way hash

Keys are derived through

the use of a one-way hash function.

Deterministic (Seeded) Wallets



Seed

a randomly generated number



combined

with other data



Data

index number

“chain code”

21 Deterministic (Seeded) Wallets



Recovery

the seed is sufficient

to recover all the derived keys



Single backup

a single backup

is sufficient



Seed

is enough

for a wallet export or import



Easy migration

between different wallet implementations

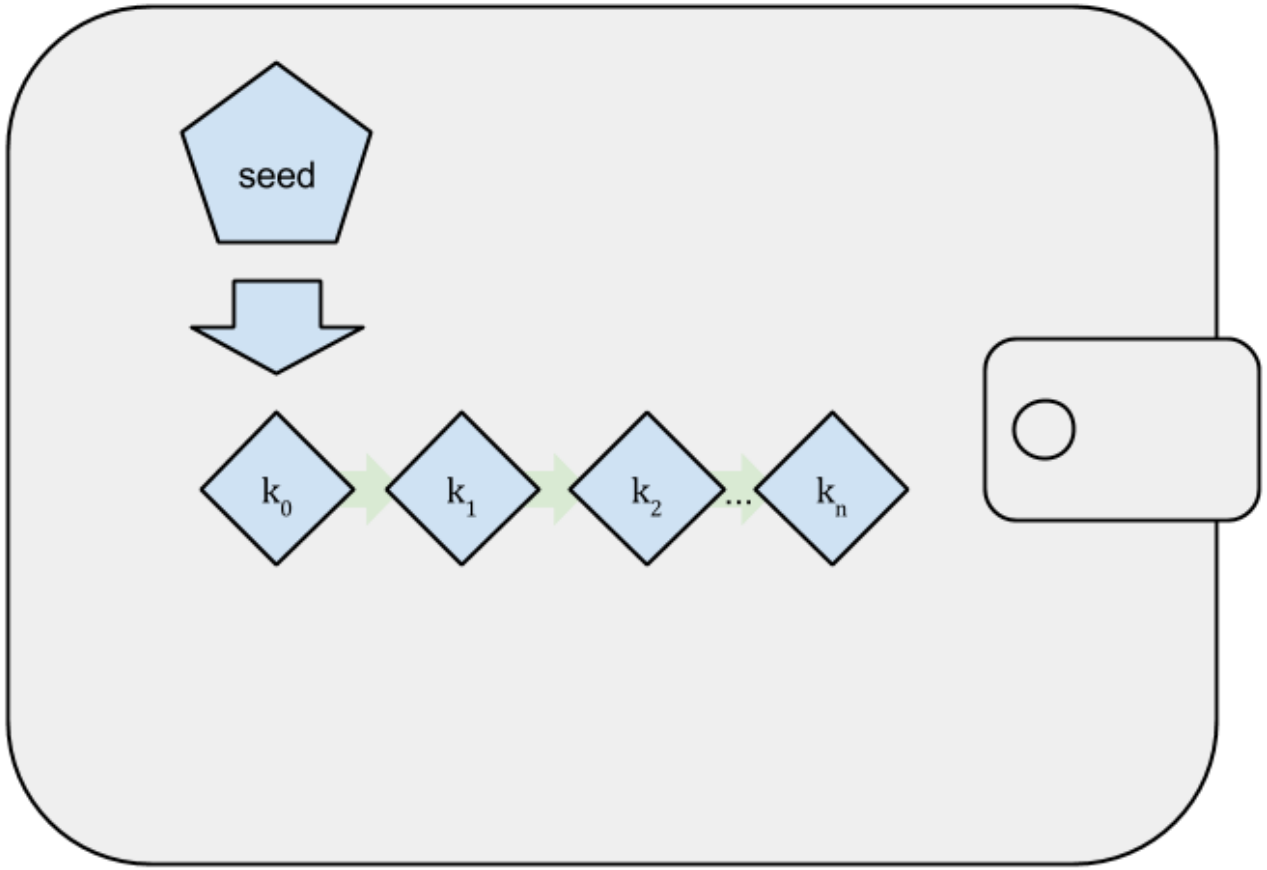


Figure 5-2. Type-1 deterministic (seeded) wallet: a deterministic sequence of keys derived from a seed

HD Wallets (BIP-32/BIP-44)



BIP-32 standard

HD wallet

advanced form of deterministic wallets



Key Hierarchy

HD wallets contain keys

derived in a tree structure,



Parent key

derive a sequence of children keys



Child key

derive a sequence of grandchildren keys successively



Infinite depth

to an infinite depth

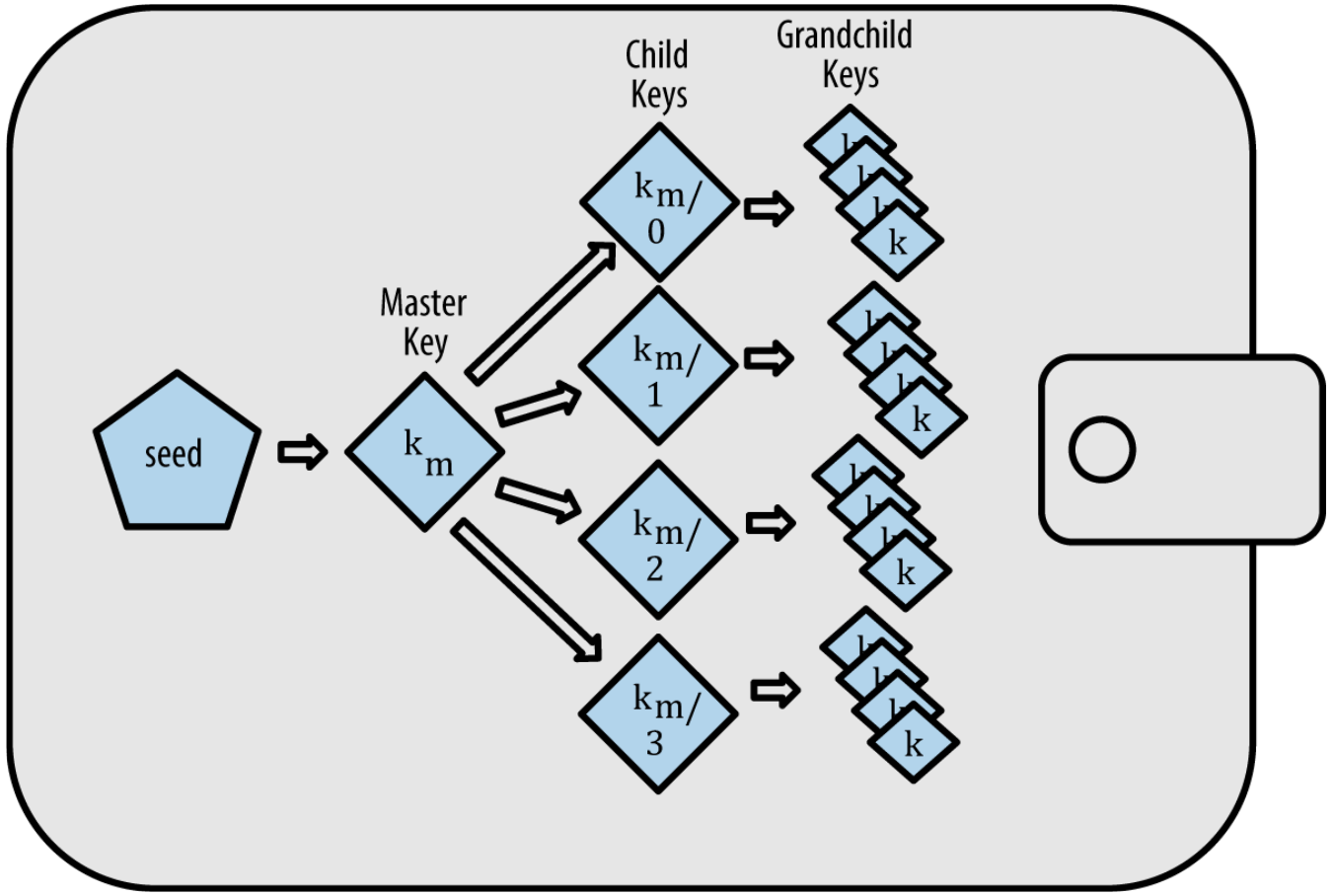
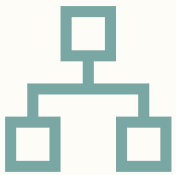


Figure 5-3. Type-2 HD wallet: a tree of keys generated from a single seed

Advantages of HD Wallets



Organizational use

Tree structure

express organizational meaning



Branches for

incoming/outgoing payments

other corporate settings



Corporate settings

departments subsidiaries

specific functions

accounting categories

Advantages of HD Wallets



Production public keys

create sequence of public keys

without accessing **private keys**



Secure payment receive

A different **public** key per transaction

on insecure servers



No spending of server

it doesn't have the private keys

so cannot spend the fund





BIP-39-Seeds and Mnemonic Codes

BIP-39: Mnemonics

using a sequence of English words to create seeds

Popularity

BIP-39: most common standard for bitcoin wallet implementation other cryptocurrencies

interoperable mnemonics

to import and export seeds for backup and recovery

28 Seed Mnemonics



Compare

Which of usecases are easier?



Random seed

0C1E24E5917779D297E14D45F14E1A1A



Mnemonic

army van defense carry jealous true garbage claim echo media make crunch



Use cases

to transcribe

record on paper

read without error

export

and import into another wallet

Wallet Best Practices

BIP Standards

BIP-32

BIP-39

BIP-43

BIP-44

Standarts help

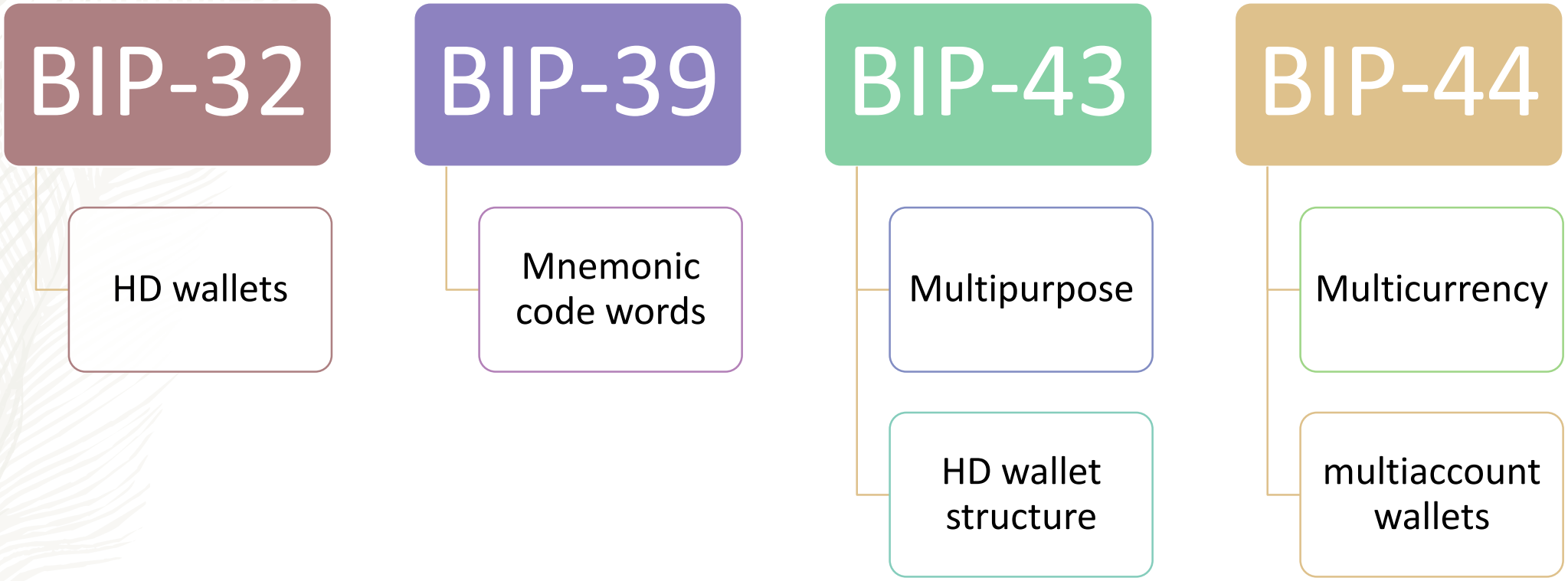
wallet interoperability

easy usage

more secure environment

increase the flexibility

30 Wallet Implementation Standards



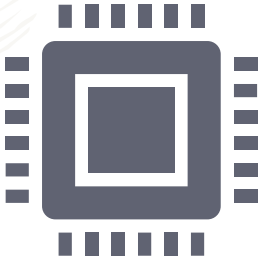
Examples

Software Wallets	supporting the standards
	Breadwallert Github
	Copay
	Mycelium
Hardware Wallets	supporting the standards
	Keepkey
	Ledger
	Trezor





Standards for Implementation



Implementation

bitcoin wallet

should be built as a HD wallet



seed for backup

encoded as mnemonic code

for backup

33 Hardware Wallet

Trezor	Device	Usage	Compliancy	Independence
• bitcoin hardware wallet • to securely bitcoin	• simple USB device • with two buttons	• stores keys in HD • signs transactions	• Trezor is compliant to Standards	• user is not reliant on • any proprietary technology • or single vendor solution



Trezor Usage



Start

for the first time the device

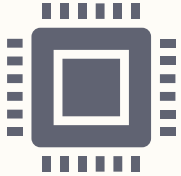
generates a mnemonic and seed



From

a built-in

hardware random number generator

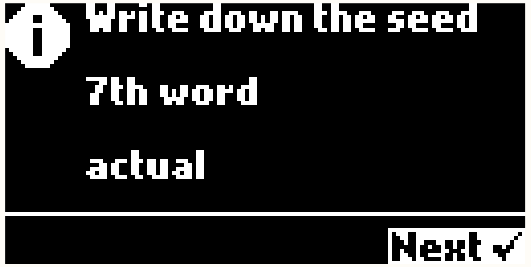


Mnemonics

the wallet displays

a numbered sequence of words

on the screen

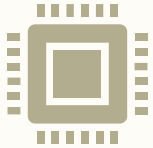


35 Gabriel Case



Create a Backup

writing down the mnemonic



Mnemonic for recovery

in the case of loss or damage

for recovery



Recovery

in a new Trezor

or in compatible wallets



Sequence is important

Preserve Sequence of words

each word in the numbered space

Table 5-1. Example paper backup of the mnemonic

1.	*army*	7.	*garbage*
2.	*van*	8.	*claim*
3.	*defense*	9.	*echo*
4.	*carry*	10.	*media*
5.	*jealous*	11.	*make*
6.	*true*	12.	*crunch*

24 Words

Most hardware wallets generate more secure 24-word mnemonic.

Usage

The mnemonic is used in exactly the same way, regardless of length.

Wallet Technology Details



Mnemonic Code Words (BIP-39)



Creating an HD Wallet from the Seed



Using an Extended Public Key on a Web Store

BIP-39: Mnemonic Code Words



Mnemonic code words

Mnemonic code words are word sequences to represent (encode) a random number



Random seed

used as a seed to derive a deterministic wallet.

BIP-39: Mnemonic Code Words



re-create the keys

and all the derived keys

from the sequence of words



of Words to show

12 to 24 words

at first creation of wallet

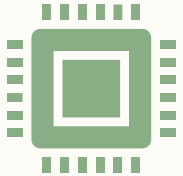
BIP-39: Mnemonic Code Words



Backup

The sequence of words

is the wallet backup

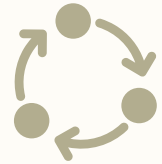


Recovery

sequence of words

can be used to recover and re-create

all the keys



Options

in the same technology

or any compatible wallet

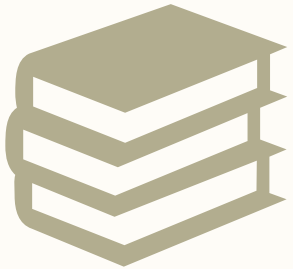
BIP-39: Mnemonic Code Words



Mnemonic words

easier for users

to back up wallets



random seeds vs mnemonic

easy to read

and correctly transcribe

42 «Brainwallets» vs «Mnemonics»



Brainwallets vs Mnemonic words

often confused with “brainwallets.”



Brainwallet

consists of words chosen by the user



Mnemonic words

are created randomly by the wallet

«Brainwallets» vs «Mnemonics»



Security

mnemonic words

much more secure



Reason

humans are very poor sources

of randomness

BIP-39 vs Electrum Wallet

BIP-39

was proposed by the company

behind the Trezor hardware wallet

BIP-39 vs Electrum

is incompatible with

Electrum's implementation.



Electrum

a different standard

with a different set of words

De facto standard

BIP-39 is achieved broad industry support

dozens of interoperable implementations

Generating mnemonic words



Automated

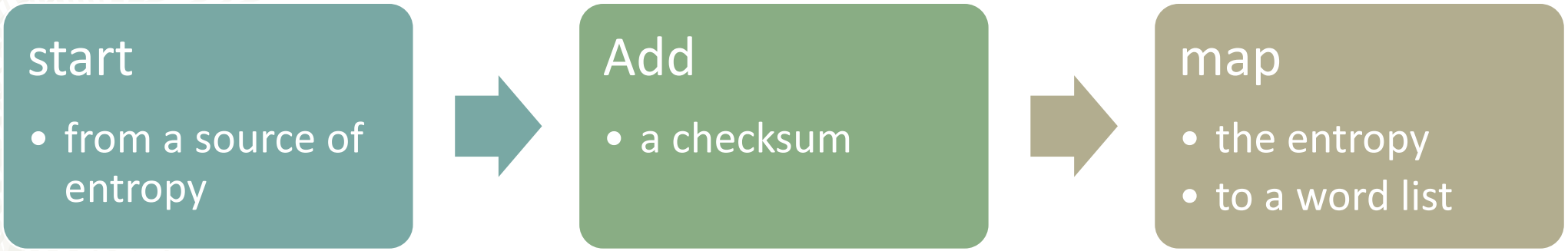
Mnemonic words automatically generated by the wallet



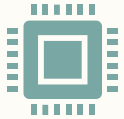
BIP-39 Compliant

Word generation Compliant to BIP-39

Steps of mnemonic word generation



47 Generating mnemonic words



Random sequence

Create a random sequence

from 128 to 256 bits



Checksum

SHA256 hash of sequence

Take its first 32 bits

Add to the end of the sequence



11 bits sections

Divide the sequence

into sections of 11 bits.



2048 words

Map each 11-bit value

to a word

from a predefined 2048 words.



2^{11}

=2048

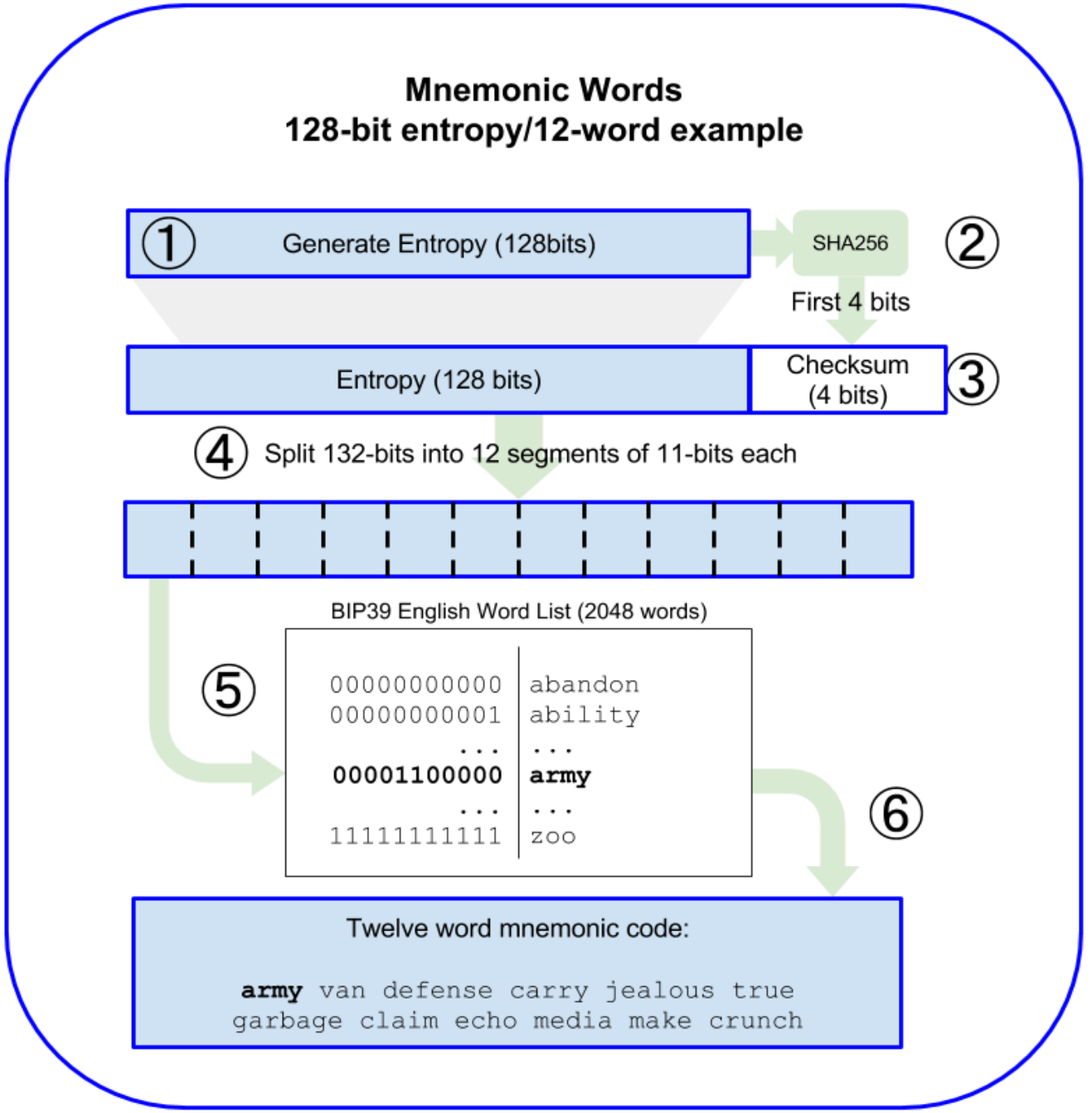


Figure 5-6. Generating entropy and encoding as mnemonic words

Table 5-2. Mnemonic codes: entropy and word length

Entropy (bits)	Checksum (bits)	Entropy + checksum (bits)	Mnemonic length (words)
128	4	132	12
160	5	165	15
192	6	198	18
224	7	231	21
256	8	264	24

From mnemonic to seed



Mnemonic words

words represent entropy

with a length of 128 to 256 bits.



Seed

The entropy is used

to derive a 512-bit seed



Keys from Seed

derive HD wallet keys

from the seed



Key stretching

PBKDF2: **P**assword-**B**ased **K**ey **D**erivation **F**unction

key-stretching function

From mnemonic to seed



key-stretching function

takes two parameters:

the mnemonic and a salt



Salt

make it difficult

to build a lookup table



Passphares

additional security factor

protecting the seed

Next Steps in “Generating mnemonic words”



PBKDF2

stretches the mnemonic and salt parameters



HMAC-SHA512

2048 rounds of hashing



Produces

a 512-bit value as its final output



The Seed

That 512-bit value is the seed.

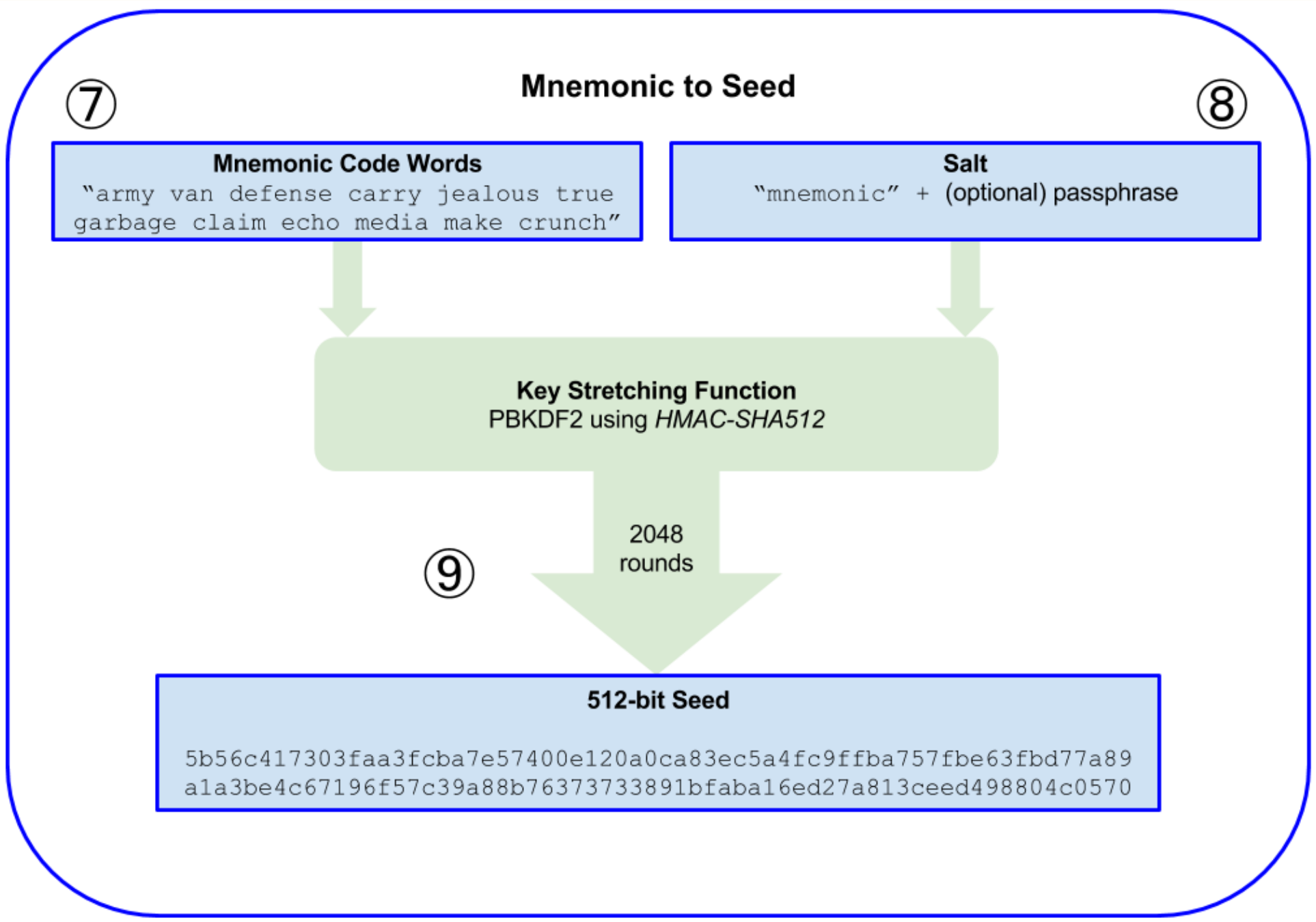


Figure 5-7 shows how a mnemonic is used to generate a seed.

54 Protection



key-stretching function

to protect against brute-force attacks

2048 rounds of hashing



Against breach

the mnemonic

or the passphrase



Cost of Attack

makes it computationally costly



of Possible Seeds

is 2^{512}

Optional passphrase in BIP-39



The BIP-39

Standard allows use of optional passphrase



If no phrase

the mnemonic is stretched with a salt "mnemonic"



Product

producing a specific 512-bit seed from any given mnemonic

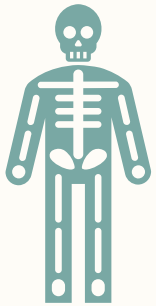
Optional passphrase in BIP-39



Passphrase affect

a passphrase leads to

a different **seed**



Seed entropy

every possible passphrase

leads to a different seed

Passphares



passphrase

creates

two important features



Protect backup

protect mnemonic backups

from compromise by a thief



Distract attacker

from the “real” wallet

containing the majority of funds



“duress wallet”

wallet with small amount of funds

Risks of Passphrase



Risks

use of passphrase

introduces risks of loss



Impact of loss

the seed is useless

funds in the wallet are lost forever

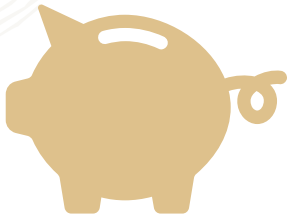


Owner’s safety

wallet owner is incapacitated

or dead

Risks about Passphrase



Backup together

the passphrase might be backed up

in the same place as the seed



No gain

it defeats the purpose of

a second factor protection

Backup & Recovery



Survival

to recover the

cryptocurrency estate



inheritors

family

other parties

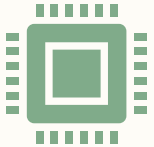
Working with mnemonic codes



Libraries

BIP-39 is implemented as a library

in many different programming languages:



python-mnemonic

reference implementation SatoshiLabs team

of the BIP-39 standard



bitcoinjs/bip39

JavaScript BIP-39 implementation

as part of the bitcoinJS framework



libbitcoin/mnemonic

BIP-39 implementation in C++

as part of Libbitcoin framework

Working with mnemonic codes



BIP-39 generator

testing

experimentation



Web page

implemented

in a standalone web page



Access

https://iancoleman.io/bip39/

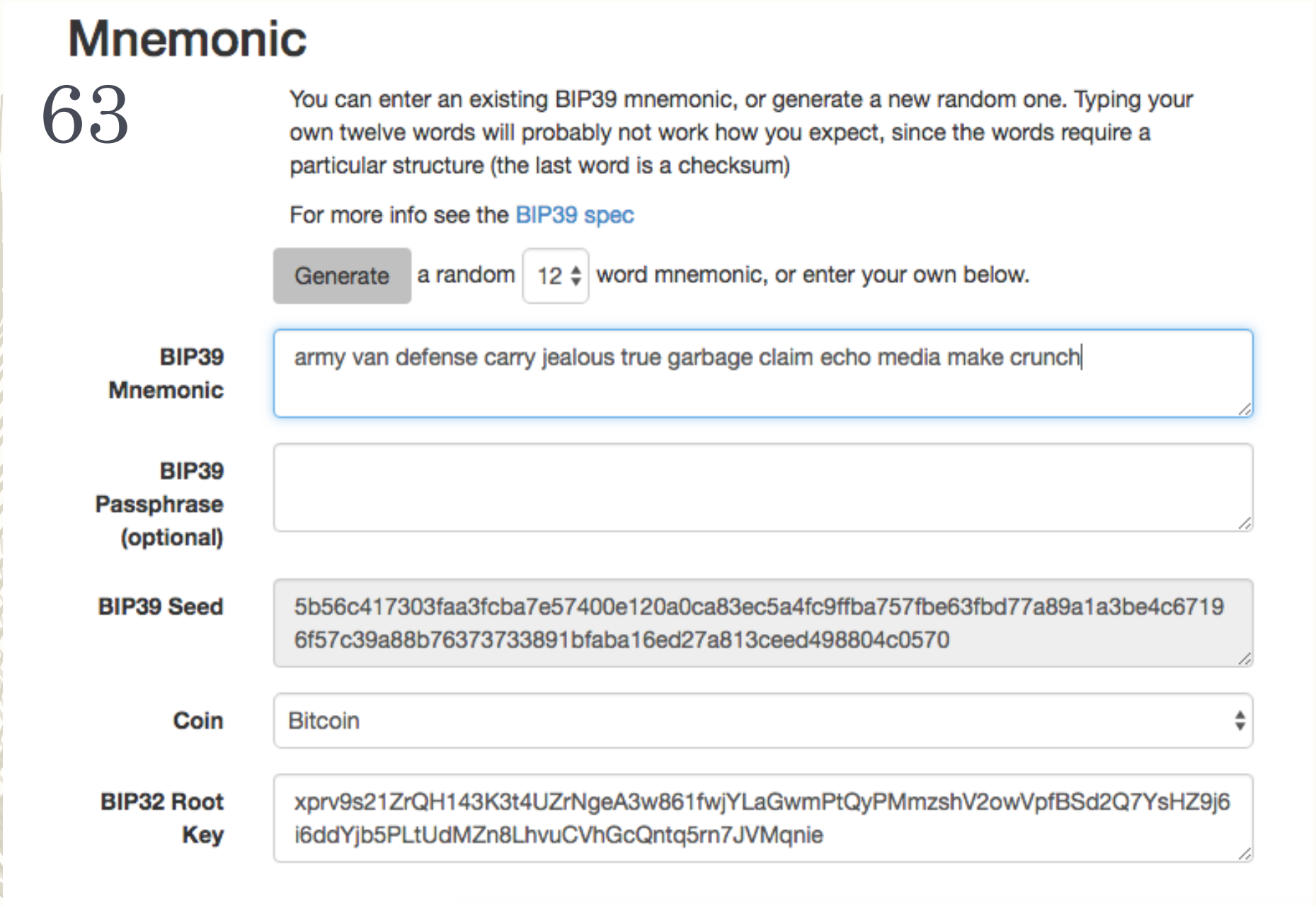


Figure 5-8. A BIP-39 generator as a standalone web page

Creating an HD Wallet from the Seed



HD wallets

created from

a single *root seed*



Root seed

random number

generated from a *mnemonic*



Root seed size

128

256

512-bit

65 Creating an HD Wallet from the Seed







Mnemonic to Root seed

the root seed

is derived from mnemonic

Root Seed to Keys

Every key in the HD wallet is

derived from the root seed

Recreation

any compatible HD wallet

can **recreate** the entire wallet



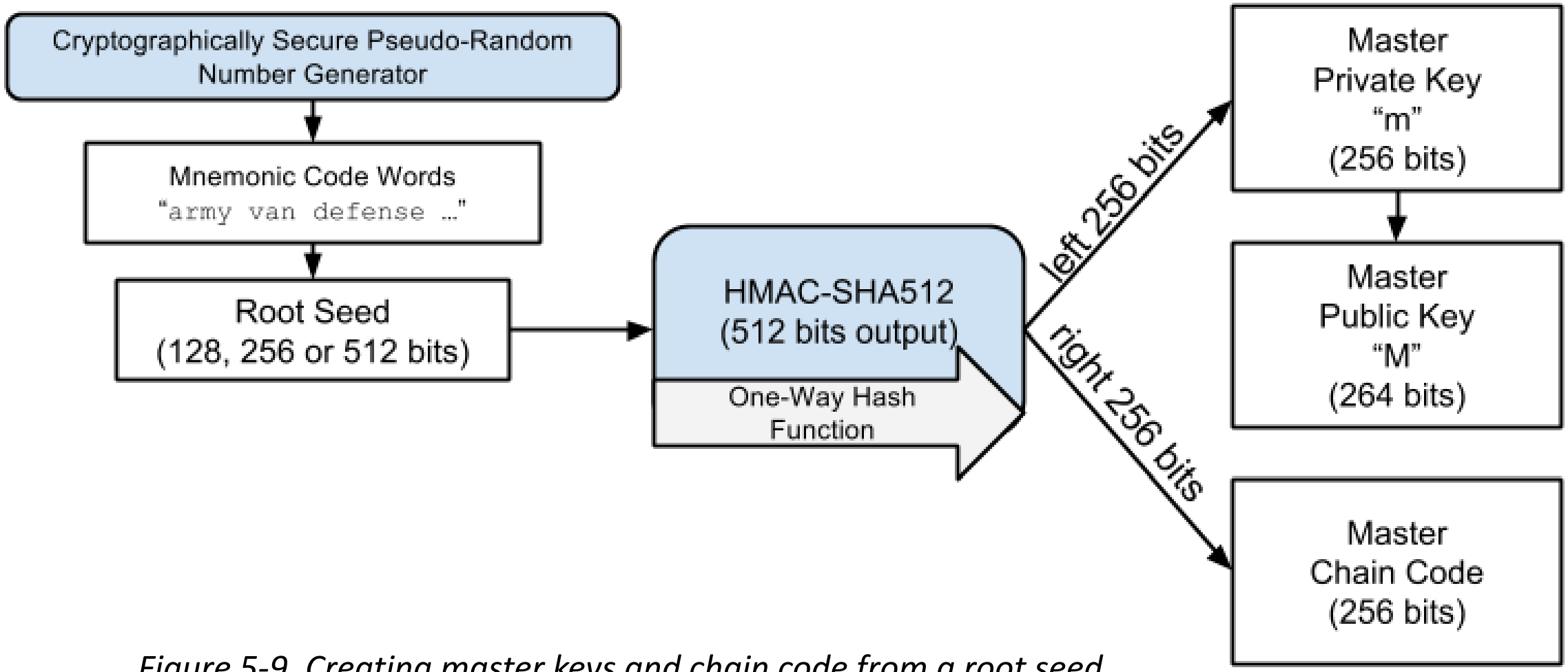
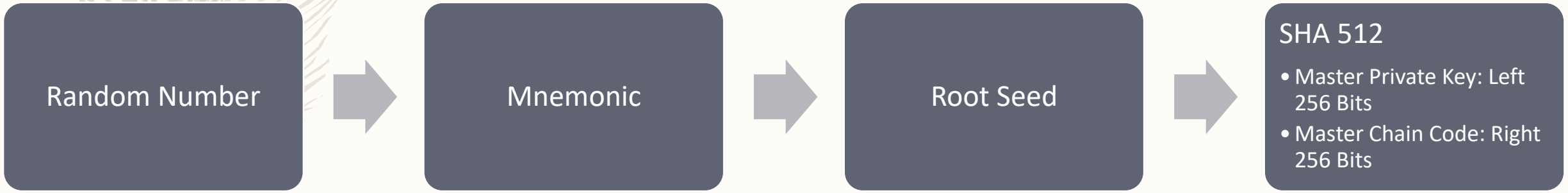


Figure 5-9. Creating master keys and chain code from a root seed



67 Root Seed, Master Key, Chain Code

The root seed

input into the HMAC-SHA512 algorithm

Result

a *master private key*

a *master chain code*.

master private key

m

to generate master public key

master public key

M = m * G

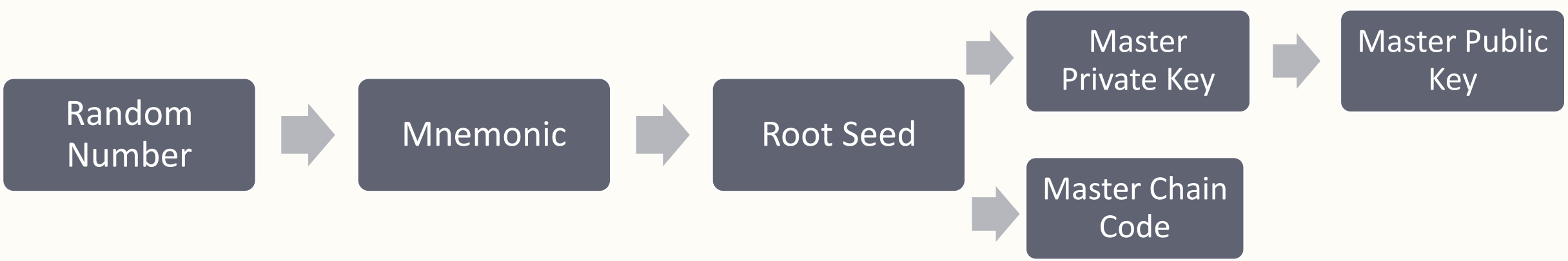
elliptic curve multiplication process

The chain code

c

to introduce entropy in the function

to create child keys from parent keys



Private child key derivation



Child keys

derived from parent

using a *child key derivation* function



Child Key Derivation function

based on **one-way hash** function



CKD Combines

Parent

Seed

Index



A Parent

private key

or public key



Seed

a chain code

256 bits

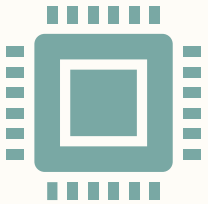


Index

index number

32 bits

Chain Code



chain code

to introduce

deterministic random data



Secure the children

the index and child key is not sufficient

to derive other child keys



Secure the siblings

chain code *prevents*

to find child's sibling keys

using the child key

Chain Code



The initial chain code

at the root of the tree

is generated from the seed



Subsequent Chain Codes

derived from

parent chain codes

Children Key Production



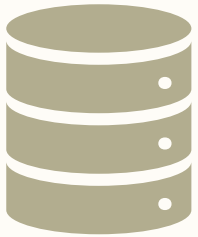
Children keys

combine

- parent key
- chain code
- index

hash

- to generate
- **children keys**



512-bit hash

combine

- **parent public key**
- **chain code**
- **index number**

hash with the **HMAC-SHA512** algorithm

- to produce a *512-bit hash*

Children Key Production



Split 512-bit hash

into two 256-bit halves



The right-half

become the **chain code**

for the child: 256 bits



The left-half

used in prodcution of

child private key



production steps

add 256 bits of the hash

to the parent private key

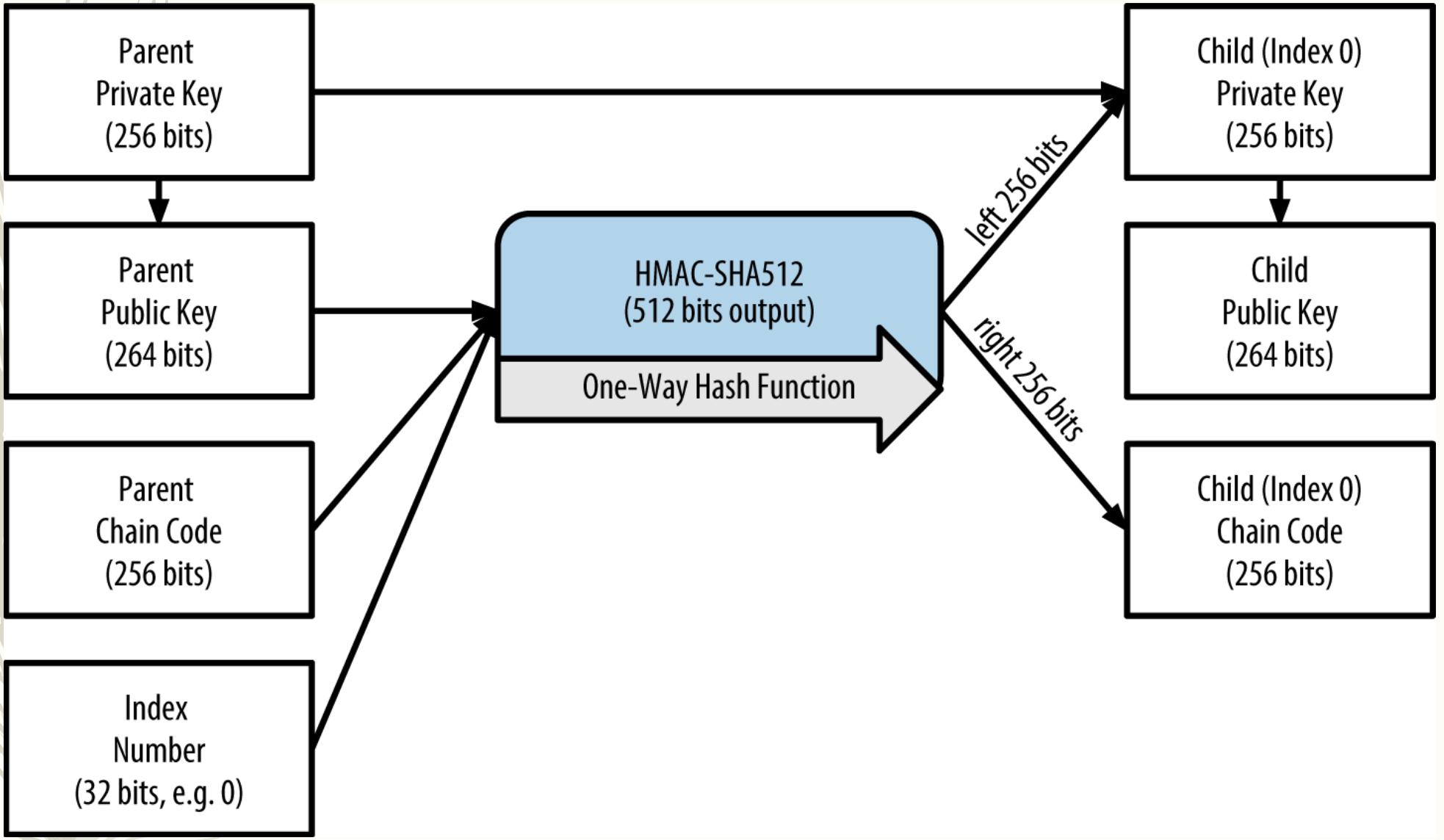


Figure 5-10. Extending a parent private key to create a child private key

Indexes



Other Children

create the other children

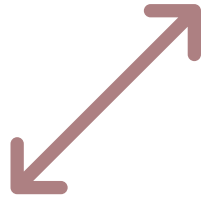
in the sequence



Steps

the index is changed

to extend the parent



Such as

Child 0

Child 1

Child 2

…

Indexes

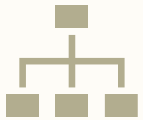


Parent capacity

2^{31} children

2,147,483,647

around 2 Billions



Leveling

Repeating the process

one level down the tree



Intermediate Node

each child can become a parent

to create its own children



of generations

in an infinite number of generations.

Usage of derived child keys

Randomness	One way function	Unreachable Parent	Inaccessible Siblings
Child private keys are random as if a random key	the derivation function is a one-way function	the child key cannot be used to find the parent key	siblings cannot be found from the child keys

Using derived child keys

Children are protected	Grandchild derivation	New branch
• parent key and chain code • can derive all the children	• the child • child chain code • are needed	• can be started with • the child private key • and the child chain code

Usage of Child Private Key



child private key

to generate

- a public key
- a bitcoin address



Signature

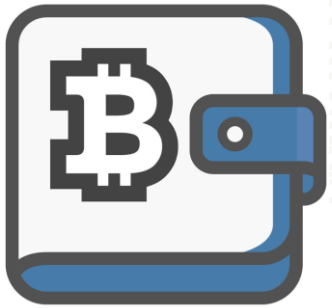
to sign transactions



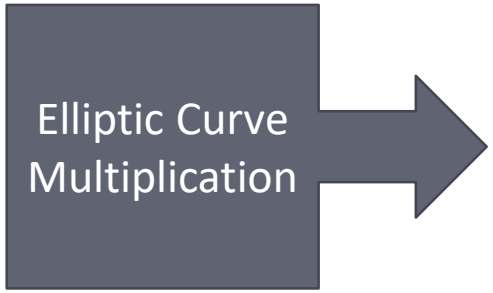
Signing

to spend anything

paid to that address













Derived Keys



Keys

Once created

Can be used as normal keys



Provide a Sequence

child private key

public key

bitcoin address



Randomness

all are indistinguishable

from random keys and addresses



Visibility

nobody can see them

outside of the HD wallet

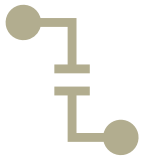
Derived Keys: Extended keys



Extended Key

Combination of

Key and Child code



Extensible keys

Can be used

to derive new children keys

at any level of tree

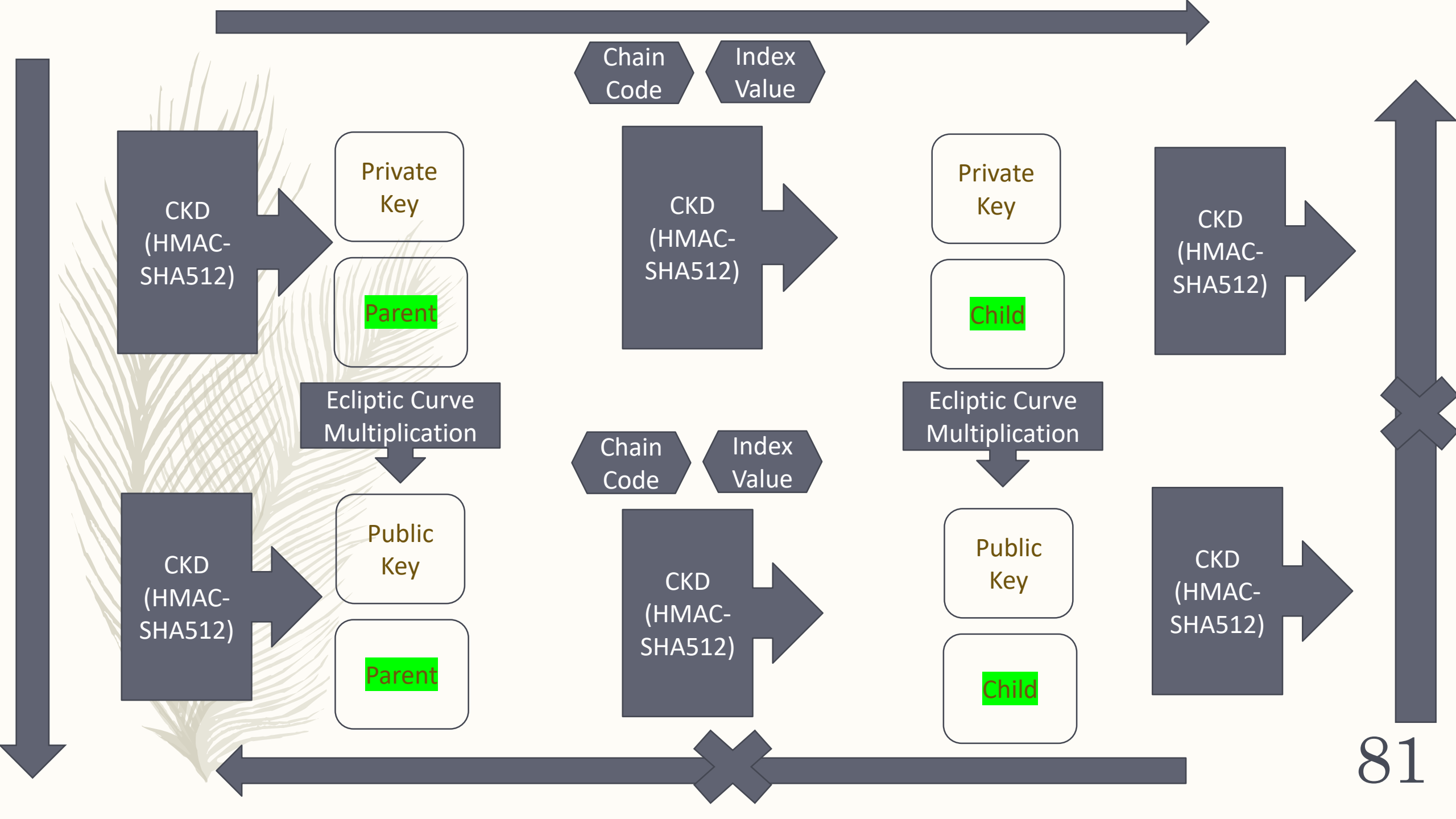


Inputs

a **key**

a **chain code**

index of the desired child

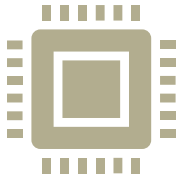

Chain Code
Index Value
CKD (HMAC-SHA512)
Private Key
Parent
Ecliptic Curve Multiplication
Public Key
Child

Key Extending



Extended keys

512 Bits



concatenation

Key: 256-bit

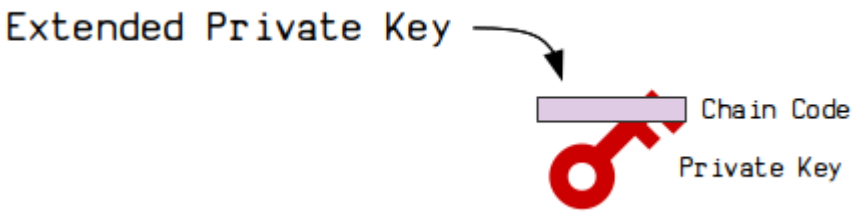
Chain Code: 256-bit

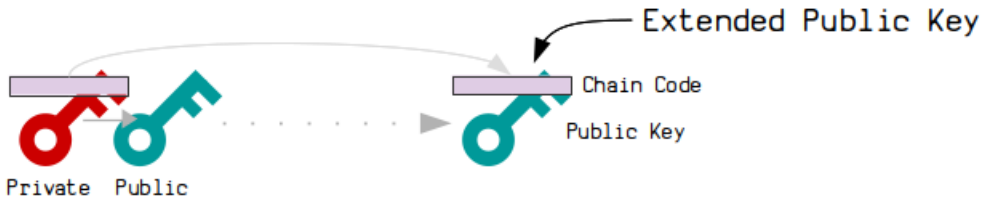


Key Types

extended **private** key

extended **public** key





https://learnmeabitcoin.com/technical/extended-keys

Extended Keys



Extended Private Key

combination of

a private key

chain code



used to

derive child private keys



Extended Public Key

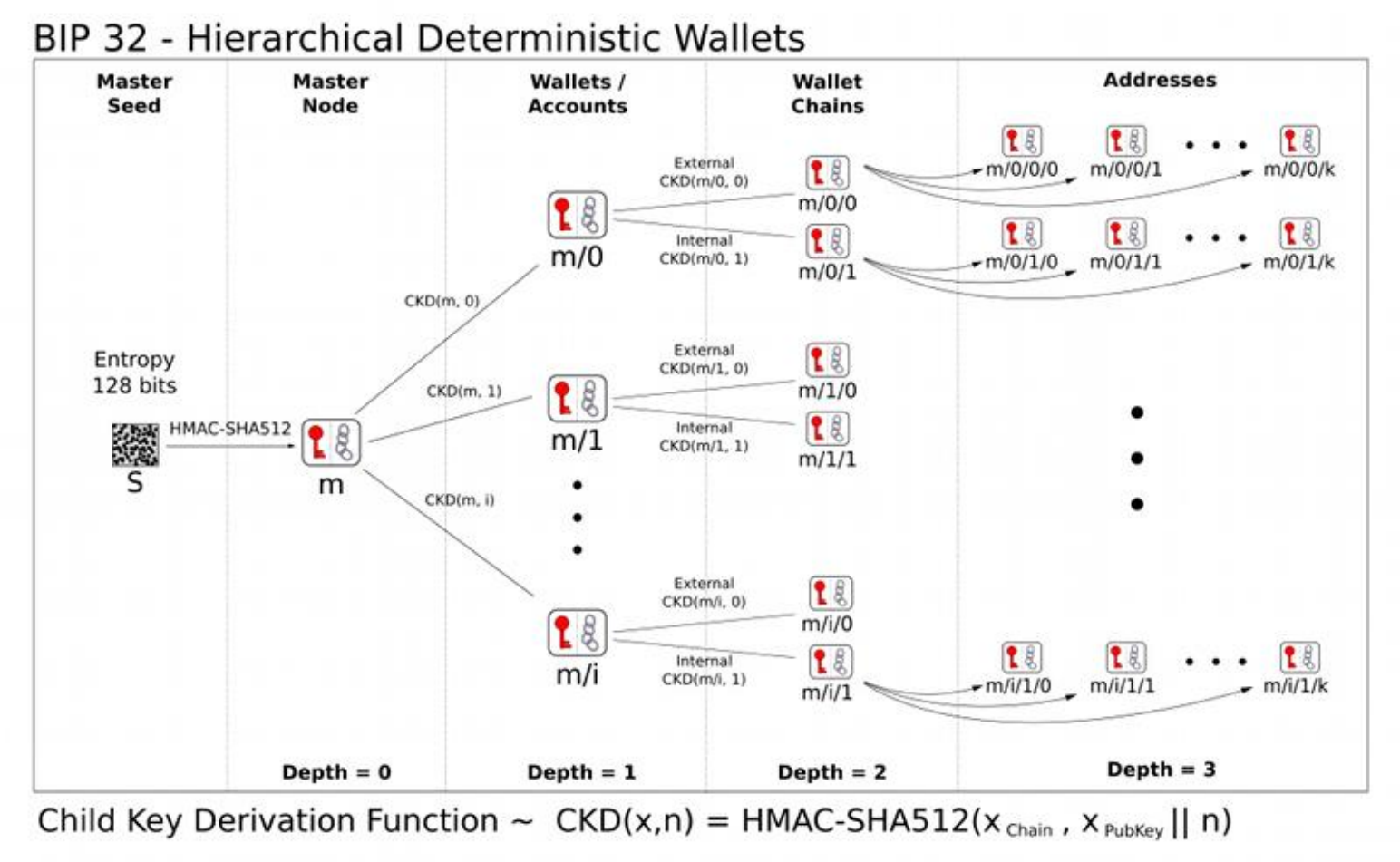
is a public key

and chain code



used to

create child public keys



Extended Key: Root of Branch

Root of Branch
in the HD wallet tree structure

Access to Rest
Access or derive the rest of the branch

Extended Key Sharing



To share

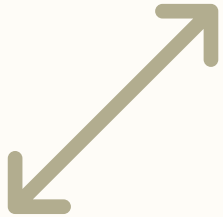
give access to the branch

share extended key



Extended key

Key + Chain Code



Generation

Extended key is enough

to derive the rest

Encoding Extended Keys



Base58Check

Extended keys

are encoded



Simplify transfer

Simplification of

export and import



Between Wallets

different

BIP-32–compatible wallets

Encoding Extended Keys: Version Number



Version Number

Uses Special Version Number



Base58Check coding

For Extended Key



prefix “xprv”

Extended private key



“xpub”

Encoded public key

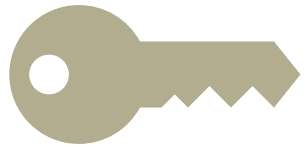
Example: Extended Private Key



Base58Check

extended *private* key

starts with **xprv**



Example

xprv9tyUQV64JT5qs3RSTJkXCWKMyUgoQp7F3h
A1xzG6ZGu6u6Q9VMNjGr67Lctvy5P8oyaYAL9CA
WrUE9i6GoNMKUga5biW6Hx4tws2six3b9c

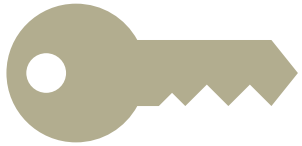
Example Extended Public Key



Base58Check

extended *public* key

starts with **xpub**



Example

xpub67xpozcx8pe95XVuZLHXZeG6XWXHpGq6Qv
5cmNfi7cS5mtjJ2tgypeQbBs2UAR6KECeeMVKZB
PLrtJunSDMstweyLXhRgPxdp14sk9tJPW9

90 Public child key derivation: Alternatives



Child Public Key

1.From child Private Key

2. From parent Public Key



1.

Child public key

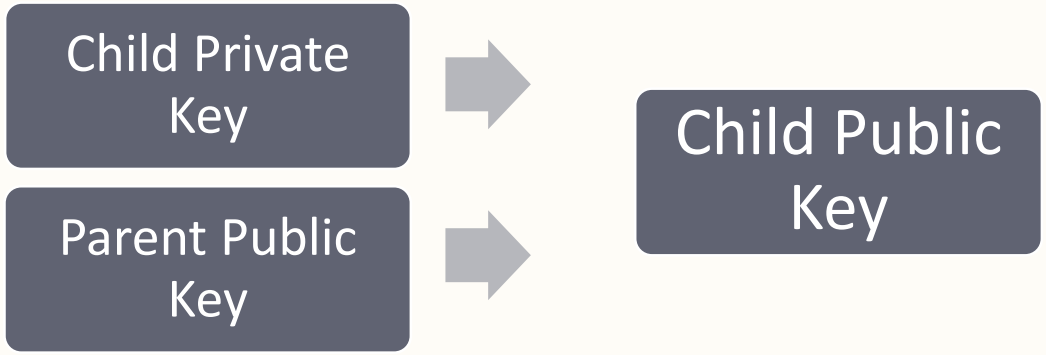
from the **child private** key

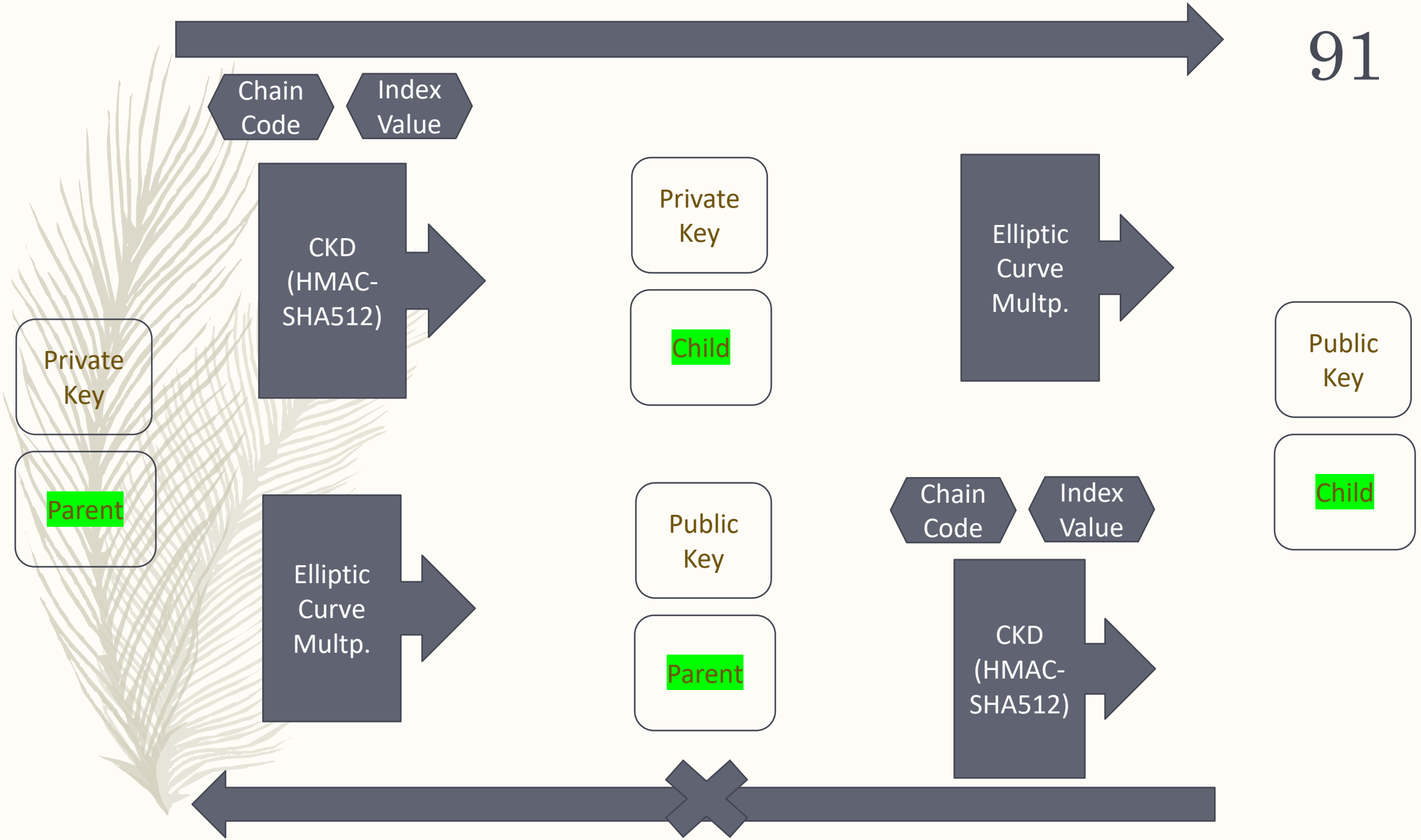


2.

from the

parent public key




Chain Code
Index Value
CKD (HMAC-SHA512)
Private Key
Parent
Private Key
Child
Elliptic Curve Multp.
Public Key
Child
Elliptic Curve Multp.
Public Key
Parent
Chain Code
Index Value
CKD (HMAC-SHA512)

1.From Parent Public Key: Method

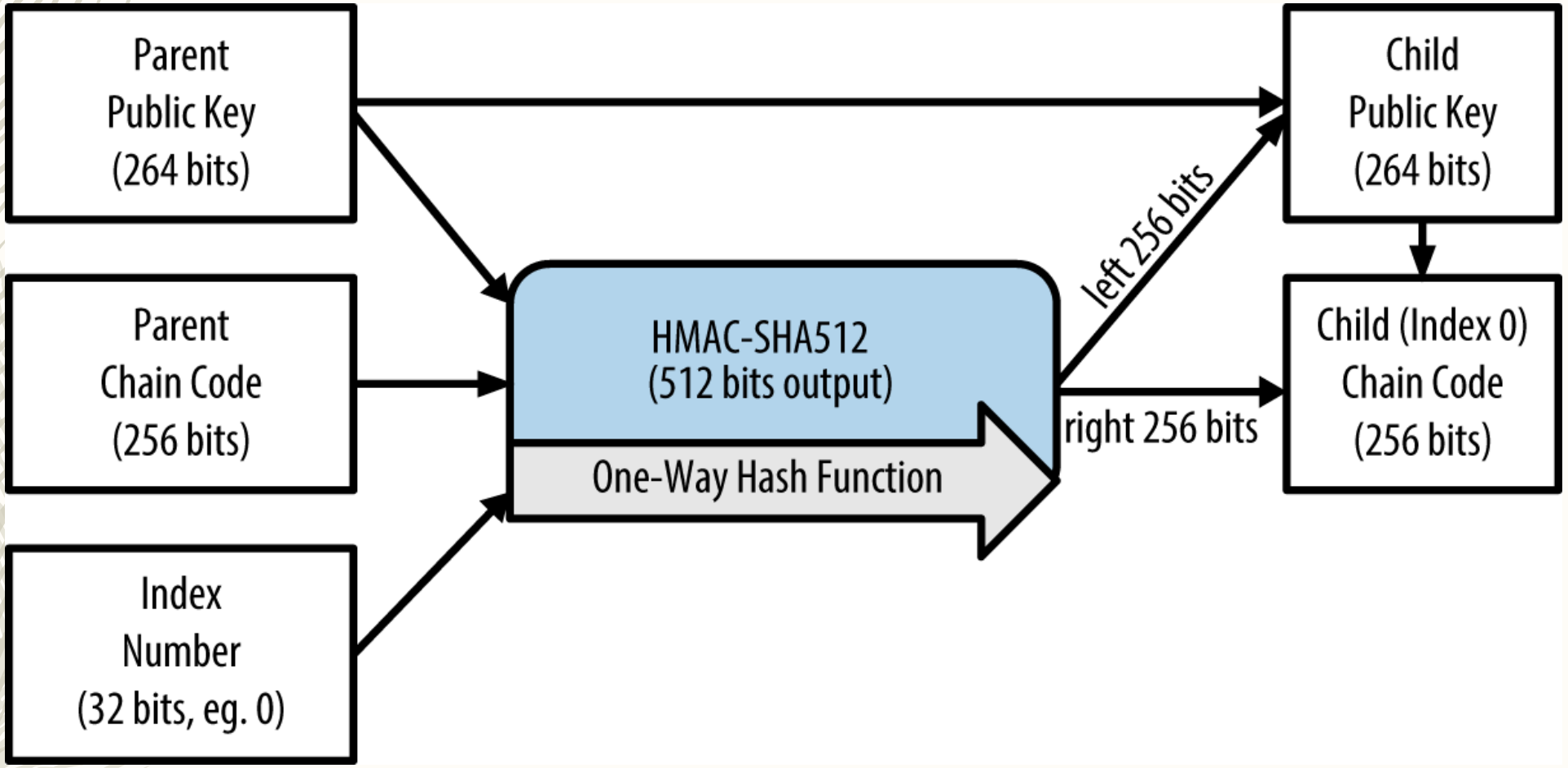


Figure 5-11. Extending a parent public key to create a child public key

1. From Parent Public Key: Protection



Secure deployment

creates very secure

public key–only deployments



No private keys

a server or application

no private keys

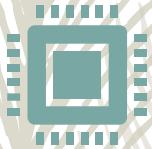


Only public keys

extended public key available

on server or application

94 1. From Parent Public Key: E-Commerce



E-Commerce Solution

install an extended public key

on a web server



Key Reproduction

use the public key derivation function

can derive infinite number of public keys



Job

create new bitcoin address

for every transaction of shopping cart

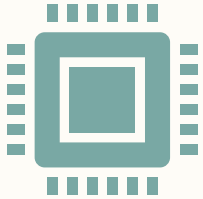


Protection

no private keys on servers

vulnerable to theft.

95 1.From Parent Public Key: Cold Storage



Cold-storage

physical storage like paper wallet

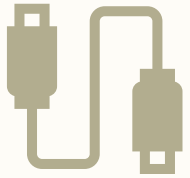
hardware wallets like Trezor



Physical storage

extended *private* key

can be physically stored



Online

Extended *Public* key

can be kept online

1.From Parent Public Key: Receving Funds



Fund Receiving

The user can create

“receive” addresses at will

while the private keys are safely stored offline.



Fund Spending

To spend the funds

the user can use the extended private key

on an offline signing bitcoin client

or sign transactions on the hardware wallet device

1.From Parent Public Key: Gabriel's Case



HD Wallet on Web Store

Let's see how HD wallets are used

by continuing our story with Gabriel's web store.



Simple Word Press Page

Gabriel first set up his web store as a hobby,

based on a simple hosted Wordpress page.



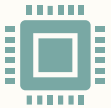
Basic Store

His store was quite basic

with only a few pages and

an order form with a single bitcoin address.

Gabriel's Case



Address from Trezor

Gabriel used the first bitcoin address generated by his Trezor device

as the main bitcoin address for his store.



Wallet controlled by Trezor

This way, all incoming payments would be paid

to an address controlled by his Trezor hardware wallet.



Order Form

Customers would submit an order using the form

and send payment to Gabriel's published bitcoin address



Trigger Email

triggering an email with the order details

for Gabriel to process.



Well for Few Orders

this system worked well enough

just a few orders each week

Gabriel's Case



New Orders

However, the little web store became quite successful

and attracted many orders from the local community.



Too Processes

Soon, Gabriel was overwhelmed.



Match Orders

With all the orders paying the same address

it became difficult to correctly match orders and transactions



The same Amount

especially when multiple orders

for the same amount came in close together.

Gabriel's Case



Better solution

Gabriel's HD wallet

offers a much better solution



Child public key

through the ability to derive public child keys

without knowing the private keys.



xpub

Gabriel can load an extended public key (xpub) on his website

to derive a unique address for every customer order.



Spend from Trezor

Gabriel can spend the funds from his Trezor,

but the xpub loaded on the website

can only generate addresses and receive funds.



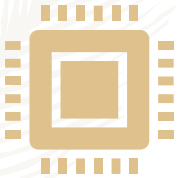
Security

This feature of HD wallets is a great security feature.

Gabriel's website does not contain any private keys

and therefore does not need high levels of security.

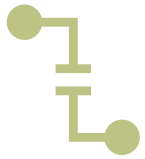
101 Gabriel's Case



Export

To export the xpub Gabriel uses the web-based software

in conjunction with the Trezor hardware wallet.



Plug in

The Trezor device must be plugged in

for the public keys to be exported.



Note

hardware wallets will never export private keys

those always remain on the device

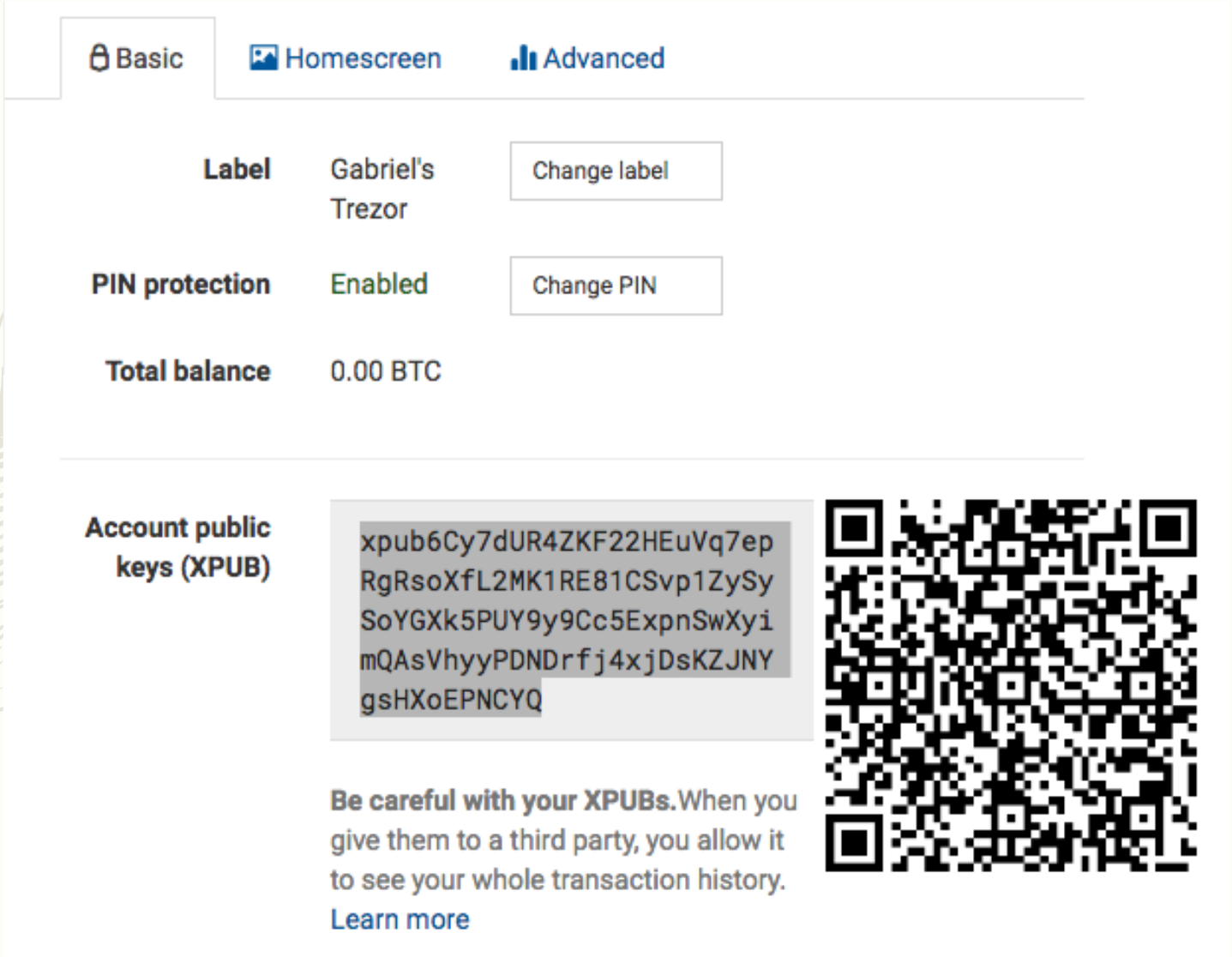


Figure 5-12. Exporting an xpub from a Trezor hardware wallet

Gabriel's Case



Copy xpub

Gabriel copies the xpub

to his web store's bitcoin shop software.



Web-Store Plugin

He uses *Mycelium Gear*

which is an open source web-store plugin

for a variety of web hosting and con tent platforms.



Unique Addresses for Purchases

Mycelium Gear uses the xpub

to generate a unique address for every purchase.

Hardened child key derivation

Useful feature

to derive a branch of public keys

from an xpub

Potential risk

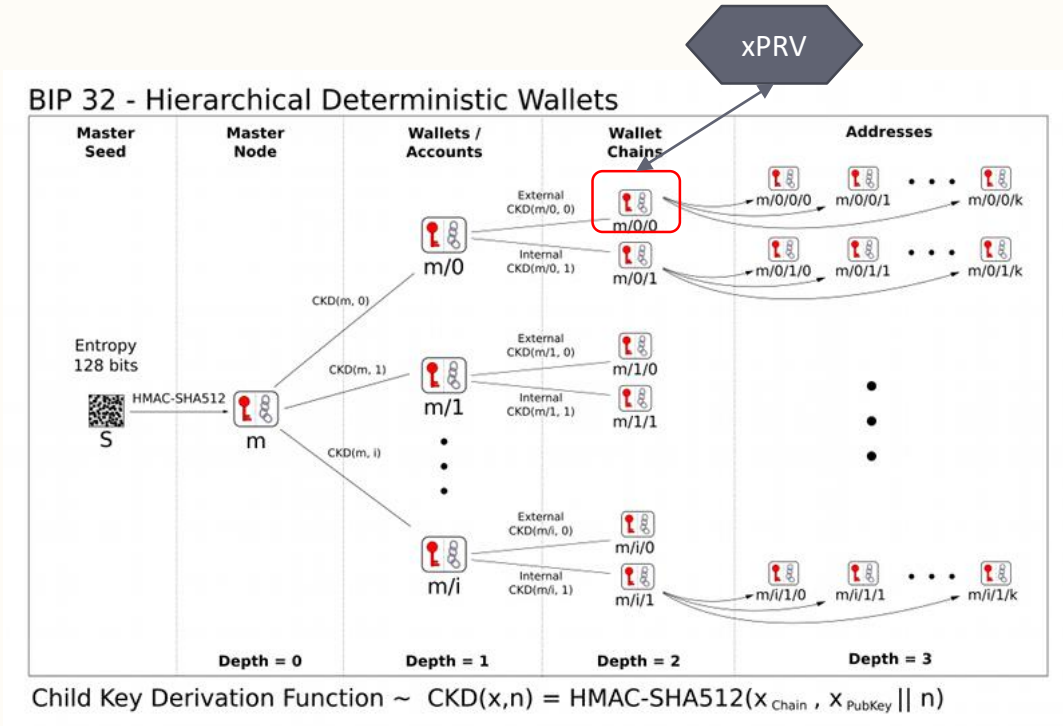
Leakage of private child key

xpub contains the chain code

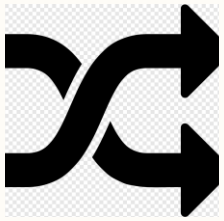
Potantial harm

the chain code

to derive all the other child private keys



Hardened child key derivation



Accident

Leakage of child private key

with parent chain code



Risk-1

All child private keys

can be deduced

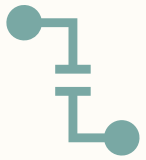


Risk-2

Parent private key

can be deduced

Hardened child key derivation



Hardened Derivation

HD wallets use

an alternative derivation function



Counter risk

Break Relationship

Parent-Child Chain



between

parent public key

and child chain code

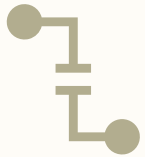
Hardened child key derivation



use the *parent private key*

to derive the *child chain code*

not the *parent public key*



Creates a Firewall

in the parent/child sequence

using the chain code



No Key Compromisation

a parent or sibling private key

cannot be compromised

Hardened child key derivation



Identical function

The hardened derivation function

looks almost identical



to the

normal child private key derivation



Difference

the *parent private key*

is used as input to the hash function

not the *parent public key*

Hardened child key derivation

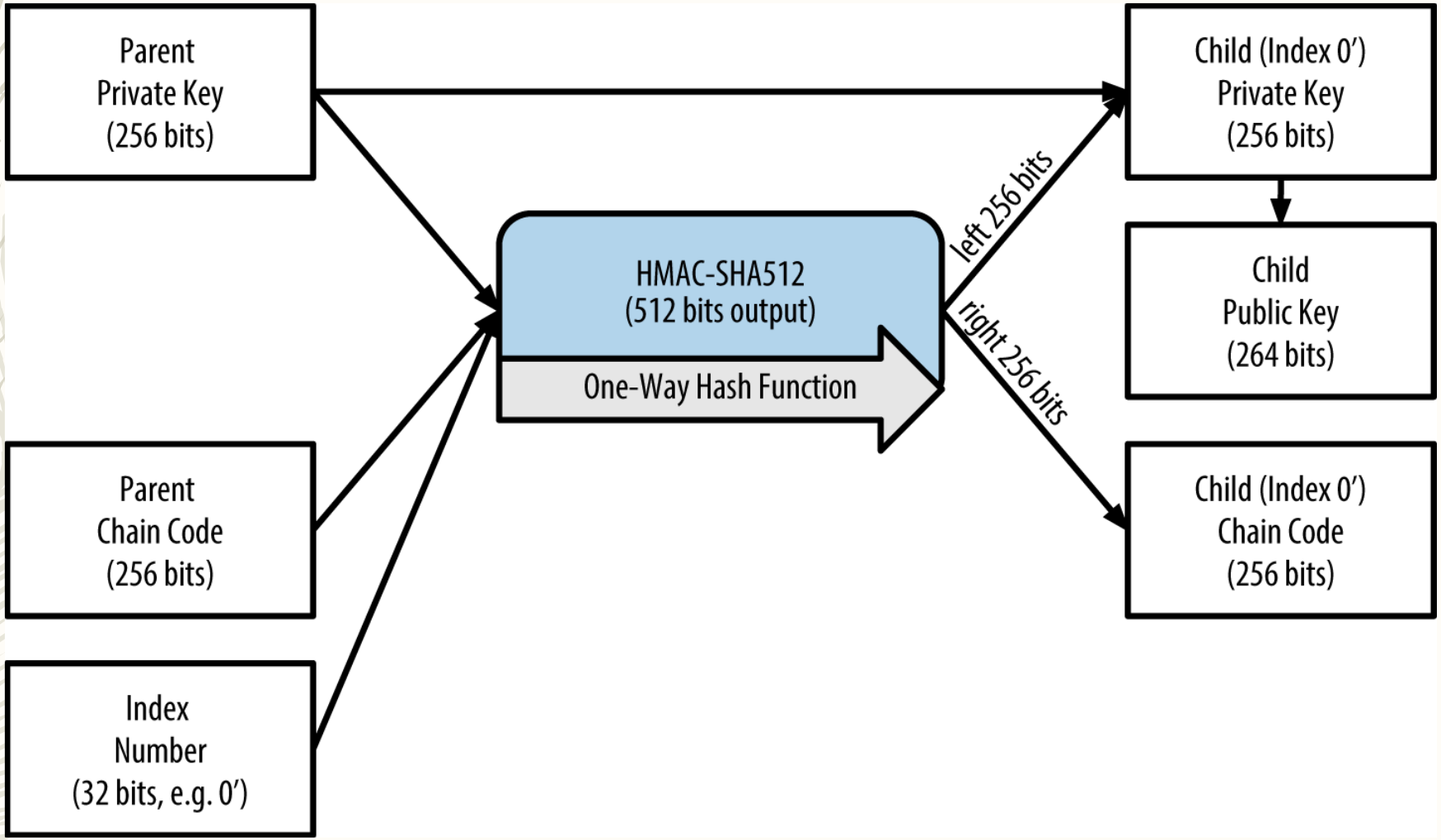
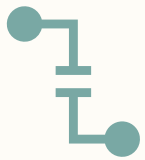


Figure 5-13. Hardened derivation of a child key; omits the parent public key

Hardening Derivation



derivation is different

from normal derivation function.



Derivation

child private key

and chain code

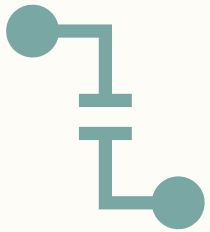


The resulting “branch” of keys

can be used to produce

extended public keys

Hardening Derivation



Protection

the chain code

cannot be exploited

to reveal any *private keys*



Reason

Hardened derivation

create a “gap” in the tree

Best Practice



Derive from Hardened Parent

rather than a normal parent



master keys level-1 children

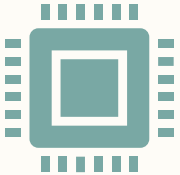
are always derived
through the hardened derivation



Reason

to prevent compromise
of the *master keys*

Index numbers for normal and hardened derivation



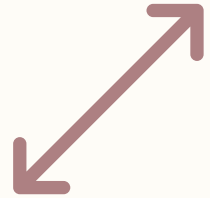
indexes: 32-Bit Integer

index numbers
in the derivation function
is a 32-bit integer



Distinguish

the normal derivation
hardened derivation



Split

two range indexes
normal derivation
hardened derivation

Index numbers for normal and hardened derivation



Normal Derivation

Index numbers

Index $<2^{31}$



between

0: 0x0

2^{31}–1: 0x7FFFFFFF



Hardened Derivation

Index numbers

Index $>=2^{31}$

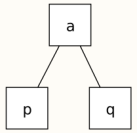


between

$2^{31:}$ 0x80000000 and

2^{32}–1: 0xFFFFFFFF

Index numbers for normal and hardened derivation



First Normal Child

displayed as **0**



First Hardened Child

displayed as **0’**

index 0x80000000



Second hardened key

be displayed as **1’**

would have index 0x80000001



and so on

2’

3’

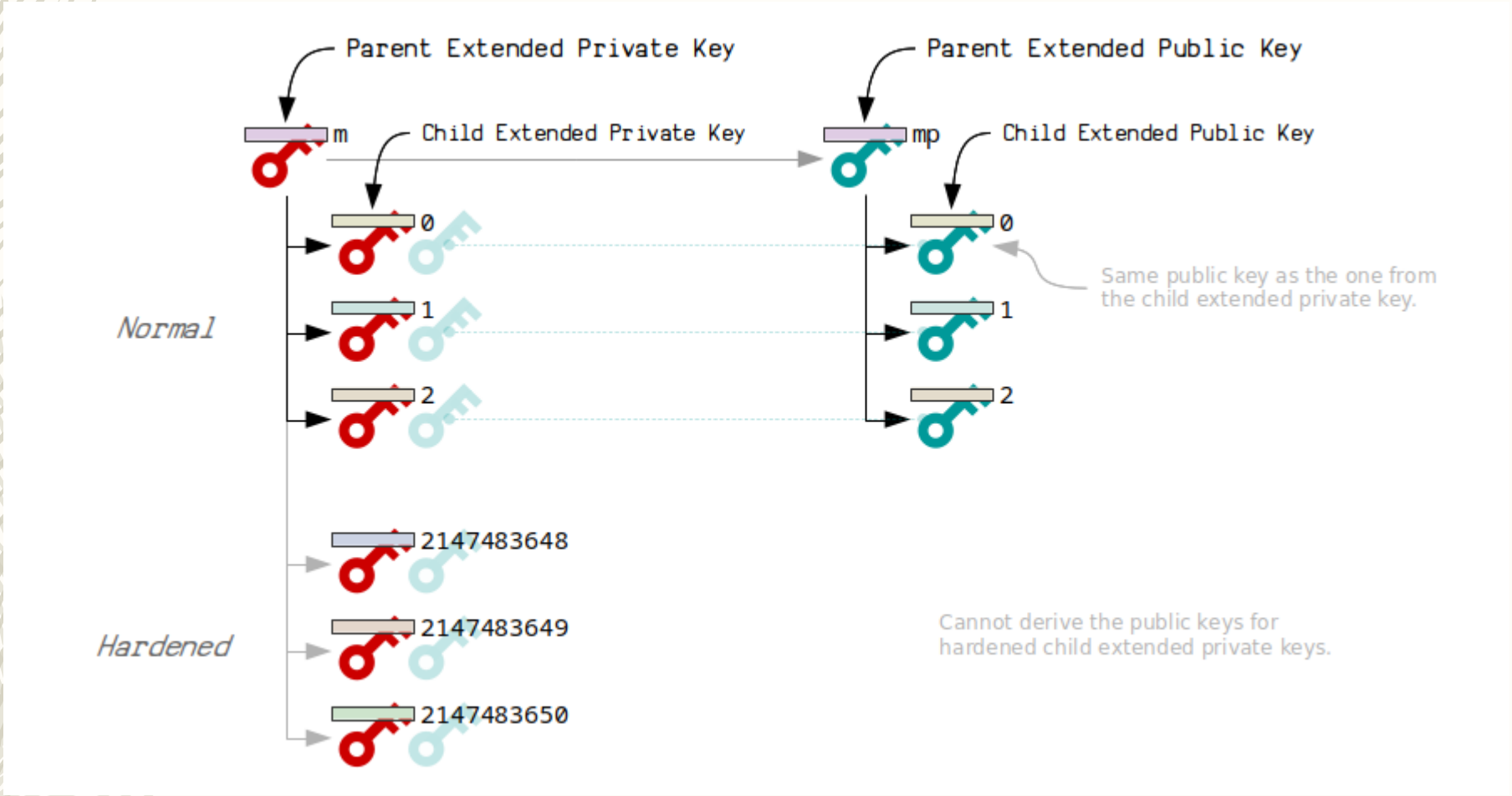
…



HD wallet index i

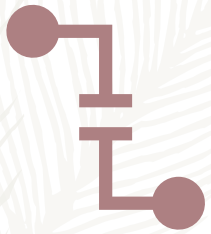
When you see an HD wallet index **i**

that means 2^{31}+i.



https://learnmeabitcoin.com/technical/extended-keys

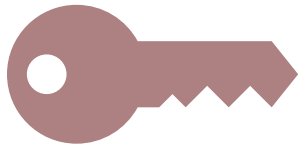
HD Wallet Key Identifier (Path)



Path Convention

Keys in an HD wallet are identified using a “path” naming convention

with each level of the tree separated by a slash (/) character.



Master Private Key

Private keys derived from the master private key

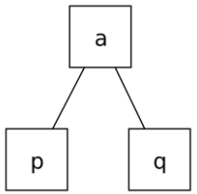
start with “m.”

HD Wallet Key Identifier (Path)



Public Keys Derived from «m»

Public keys derived from the master public key start with “M.”



First Child

private key is m/0

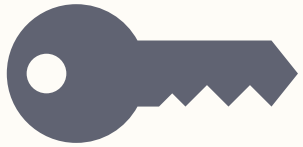
Public Key is M/0.



Second Grandchild

The second grandchild of the first child is m/0/1, and so on.

HD Wallet Key Identifier (Path)



Finding Master Key

The “ancestry” of a key is read from right to left

until you reach the master key from which it was derived.



Identifier m/x/y/z

the zth child of key m/x/y

which is the yth child of key m/x

which is the xth child of m

120 HD Wallet Key Identifier (Path)

Table 5-6. HD wallet path examples

HD path	Key described
m/0	The first (0) child private key from the master private key (m)
m/0/0	The first grandchild private key of the first child (m/0)
m/0'/0	The first normal grandchild of the first *hardened* child (m/0')
m/1/0	The first grandchild private key of the second child (m/1)
M/23/17/0/0	The first great-great-grandchild public key of the first great-grandchild of the 18th grandchild of the 24th child

Navigating The HD Wallet Tree Structure



Flexibility of HD Structure

The HD wallet tree structure offers tremendous flexibility.



4 Billion Children

Each parent extended key can have 4 billion children:

2 billion normal children and 2 billion hardened children.



Next Level

Each of those children can have another 4 billion children, and so on.

Navigating The HD Wallet Tree Structure



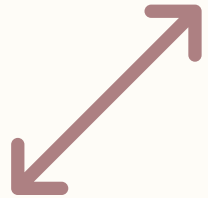
Tree Height

The tree can be as deep as you want, with an infinite number of generations.



Navigation

With all that flexibility, however, it becomes quite difficult to navigate this infinite tree.



Transfer of Wallets

It is especially difficult to transfer HD wallets between implementations,

because the possibilities for internal organization into branches and subbranches are endless.

Navigating The HD Wallet Tree Structure



Solutions

Two BIPs offer a solution to this complexity

by creating some proposed standards for the structure of HD wallet trees.



BIP-43

proposes the use of the first hardened child index

as a special identifier

that signifies the “purpose” of the tree structure.



1 Level Per Branch

Based on BIP-43, an HD wallet should use only one level-1 branch of the tree,

with the index number identifying the structure and namespace of the rest of the tree by defining its purpose.



m/i/

For example, an HD wallet using only branch m/i/

is intended to signify a specific purpose

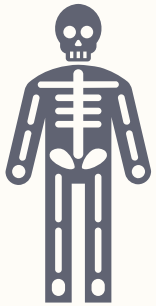
and that purpose is identified by index number “i”

Navigating The HD Wallet Tree Structure



Multiaccount Structure

Extending that specification,
BIP-44 proposes a multiaccount structure
as “purpose” number 44 under BIP-43.



BIP-44 structure

All HD wallets following the BIP-44 structure are
identified by the fact that
they only used one branch of the tree: m/44/.

125 Navigating The HD Wallet Tree Structure

1

5 Predefined Tree Levels

- BIP-44 specifies the structure
- as consisting of five predefined tree levels

2

Levels

- m / purpose
- / coin_type
- / account
- / change
- / address_index

Navigating The HD Wallet Tree Structure

The first-level	The second-level	3 Currencies	The third level	two bitcoin “accounts”
• “purpose” is always set to 44’.	• “coin_type” specifies the type of cryptocurrency coin, • allowing for multicurrency HD wallets • where each currency has its own subtree under the second level.	• Bitcoin is m/44'/0', • Bitcoin Testnet is m/44'/1', • Litecoin is m/44'/2'.	• is “account,” • which allows users to subdivide their wallets into separate logical subaccounts, • for accounting or organizational purposes.	• For example, an HD wallet might contain two bitcoin “accounts”: • m/44'/0'/0' • and m/44'/0'/1'. • Each account is the root of its own subtree.

Navigating The HD Wallet Tree Structure



The fourth level

"change," an HD wallet has two subtrees,

one for creating receiving addresses

and one for creating change addresses.



Normal derivation

Note that whereas the previous levels used hardened derivation,

this level uses normal derivation.



Usage

This is to allow this level of the tree

to export extended public keys

for use in a non-secured environment.



Fifth level

Usable addresses are derived by the HD wallet

as children of the fourth level,

making the fifth level of the tree the "address_index."



For example,

the third receiving address for bitcoin payments

in the primary account would be M/44'/0'/0'/0/2.

Table 5-7. BIP-44 HD wallet structure examples

HD path	Key described
M/44'/0'/0'/0/2	The third receiving public key for the primary bitcoin account
M/44'/0'/3'/1/14	The fifteenth change-address public key for the fourth bitcoin account
m/44'/2'/0'/0/1	The second private key in the Litecoin main account, for signing transactions

Discussions

Your opinion
What do you think on this chapter?

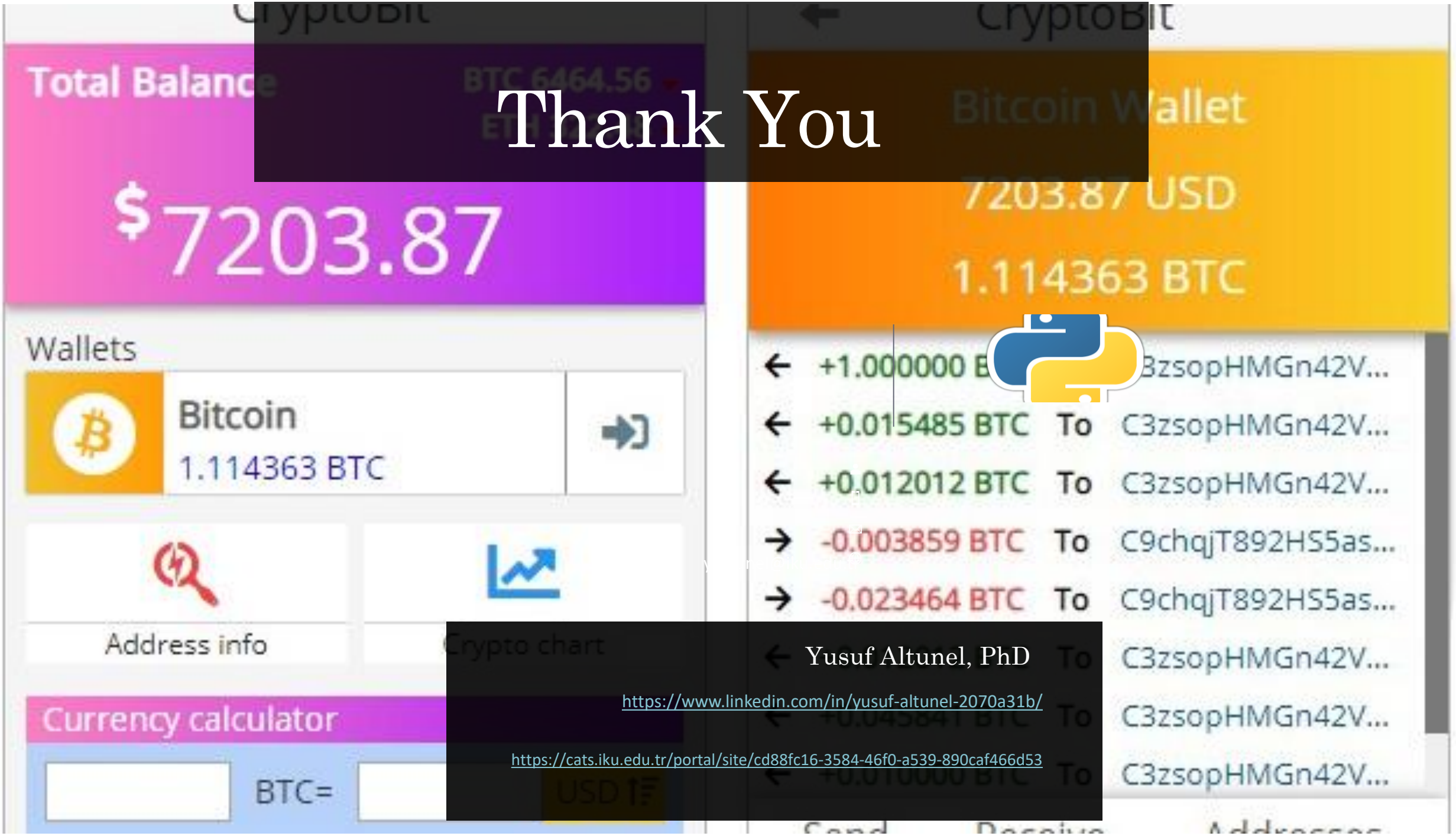
Challenging
What is the most interesting point?

Clarification
What are the unclarified points?

Critical issue
Select and explain a critical issu in this chapter



Thank You



Yusuf Altunel, PhD

https://www.linkedin.com/in/yusuf-altunel-2070a31b/

https://cats.iku.edu.tr/portal/site/cd88fc16-3584-46f0-a539-890caf466d53